

ABRA — Technical Documentation

Version 2.4.0 · Last updated 2026-07-23

Written in ASD-STE100 Simplified Technical English. Sentences are short. The voice is active. One word has one meaning. The document follows the Diátaxis structure: Tutorial, How-to, Reference, Explanation.

1. Tutorial — run ABRA for the first time

Do these steps in order.

1. Get the code. Clone the repository `github.com/willhoop/abra`.
2. Get the sibling engine. Clone `github.com/willhoop/chomp` next to it. ABRA reads it at `../CHOMP`.
3. Open the site. Open `web/index.html` in a web browser. The site needs no build step and no server.
4. Visit a model. Click a house in the town. Open **PORY** and move the sliders. The win % updates.
5. Run one check. In a terminal, run `node engine/validate_damage.js`. It confirms the damage engine.

You now have the site and one validated model.

2. How-to — common tasks

Pull new replays. `PAGES=6 CONC=20 node engine/durable-ingest.js data/games.ladder.jsonl` The command adds only new games. It never duplicates a game and never re-fetches a stored game.

Rebuild the meta model. `node engine/analyze.js data/games.ladder.jsonl` writes `data/meta-usage.json`.

Refresh the site data. `python3 engine/refresh-site-data.py` writes `data/live.js`, `data/archetypes.json`, and `data/kad-replays.js`.

Run a model or an evaluation.

- GURU matchup matrix: `python3 engine/guru.py`
- XATU belief / policy eval: `python3 engine/eval_policy.py`
- PORY value net: `python3 engine/pory.py`
- CHOMP-EV proof: `node engine/chomp_ev.js`
- SLOWKING preview-Nash (species): `python3 engine/slowking_preview.py`
- SLOWKING preview-Nash (playstyle): first `node engine/playstyle.js`, then `MATRIX_FILE=data/playstyle-matchups.json TAG=playstyle python3 engine/slowking_preview.py`

Validate the damage engine. `node engine/validate_damage.js` . It fails if any scenario is more than 5% from `@smogon/calc` .

Run the tests. `node tests/test-parse.js` , `node tests/test-dynamics.js` , `node tests/test-medicham.js` , `node tests/test-chomp-ev.js` , `python3 tests/test-jolteon.py` , `python3 tests/test-slowking.py` .

Edit the site, then mirror it. After you change `web/index.html` , copy it: `cp web/index.html app/index.html` .

3. Reference

3.1 Stored game record (`data/games.ladder.jsonl` , one JSON object per line)

Field	Meaning
<code>id</code> , <code>date</code>	replay id and upload time
<code>p1</code> , <code>p2</code>	<code>{name, rating, bot}</code> per player
<code>six.p1/p2</code>	the six revealed at preview
<code>brought.p1/p2</code>	the four actually brought
<code>lead.p1/p2</code>	the two led
<code>sets</code>	per species, the revealed moves / item / ability
<code>turns</code>	per-turn events (move, damage, faint, status, field)
<code>winner</code>	the winning name

3.2 Model outputs

File	Written by	Contents
<code>data/damage-validation.json</code>	<code>validate_damage.js</code>	damage error vs <code>@smogon/calc</code>
<code>data/guru-matchups.json</code> , <code>guru.js</code>	<code>guru.py</code>	archetype matchup matrix, Wilson CIs
<code>data/xatu.json</code> , <code>xatu.js</code>	<code>xatu.py</code>	opponent set/move belief
<code>data/pory.js</code> , <code>pory-eval.json</code>	<code>pory.py</code>	mid-game value net + its score
<code>data/chomp-ev.json</code>	<code>chomp_ev.js</code>	the bring proof (null result)
<code>data/playstyle-matchups.json</code>	<code>playstyle.js</code>	playstyle matchup matrix
<code>data/slowking-eval.json</code> , <code>slowking-playstyle-eval.json</code> , <code>slowking*.js</code>	<code>slowking_preview.py</code>	preview equilibrium + exploitability

File	Written by	Contents
<code>data/meta-usage.json</code> , <code>live.js</code>	<code>analyze.js</code> , <code>refresh-site-data.py</code>	usage model + live site counts

3.3 Continuous collection

A GitHub Action (`.github/workflows/ingest.yml`) runs the pull hourly and commits the store. A separate tests workflow runs the test suite and the damage validation on every push and pull request.

4. Explanation

Store raw, analyse on top. ABRA saves every game with every fact it may ever need. Every filter — rating tier, humans-only, archetype, playstyle — runs over the store at read time. A change to how the games are segmented is a re-computation, not a re-download. This makes the fetch a one-time cost and keeps the analysis free to change.

Support decisions, do not predict outcomes. In this format, the winner of a game is near-impossible to predict from the two team sheets; even a player-rating model ties a coin. ABRA therefore judges each model on a decision, not on the match result. Each probability ships a proper score (log-loss or Brier), a confidence interval, and an honest baseline.

Why the confidence interval is clustered. States within one game are correlated. A confidence interval that resamples states would be too narrow. ABRA resamples whole games instead. For a matchup matrix, it also resamples each cell from its Beta distribution before it solves, so the interval carries the small-sample uncertainty.

Honest negatives are kept. Two results are negative and are reported plainly: the team-picker's brings do not beat a coin (CHOMP-EV), and the playstyle rock-paper-scissors cycle rests on small samples and is suggestive, not settled. A negative that is measured is more useful than a positive that is asserted.

Companion documents. [White paper](#) · [Deck](#) · [Project summary](#) · [Model ledger](#) · [Changelog](#)